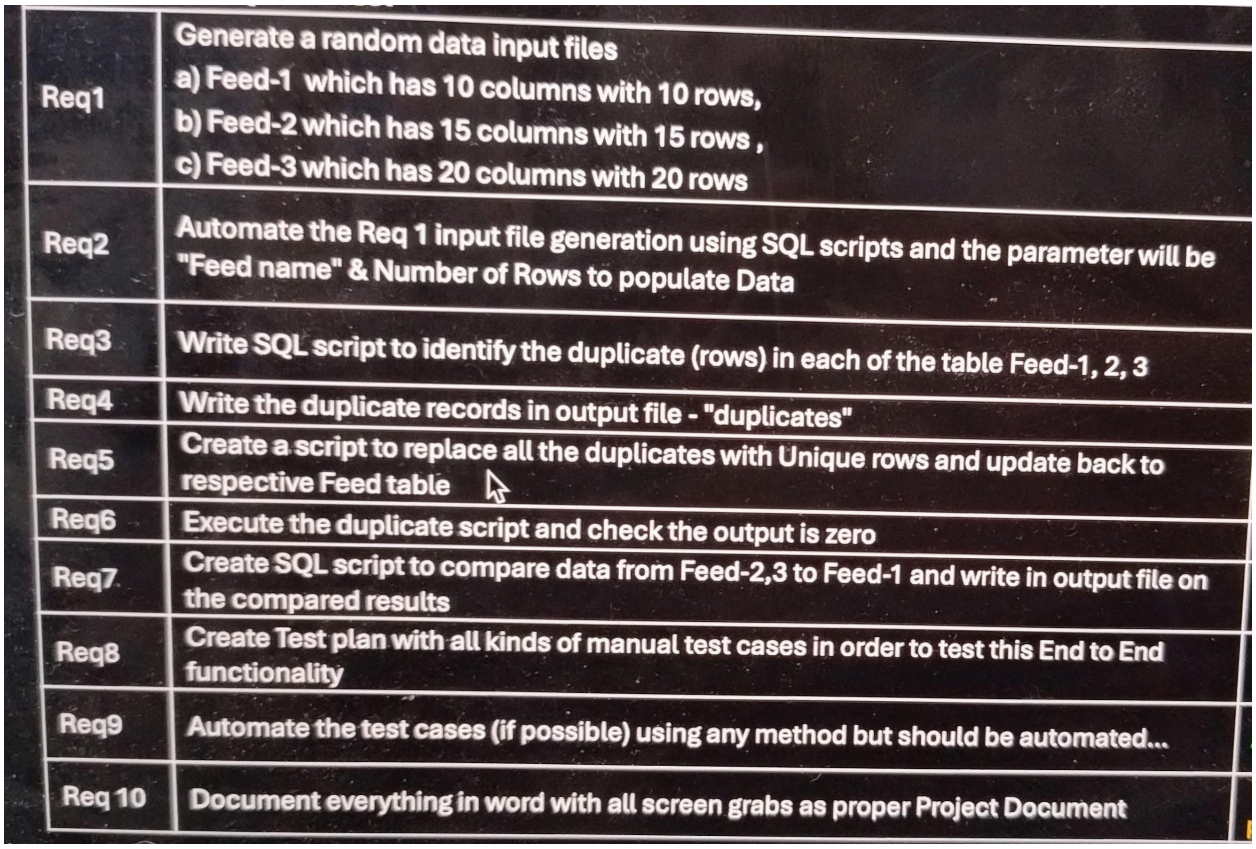
Project Document: SQL Data Quality & Comparison Framework



Req1	Generate a random data input files a) Feed-1 which has 10 columns with 10 rows, b) Feed-2 which has 15 columns with 15 rows , c) Feed-3 which has 20 columns with 20 rows
Req2	Automate the Req 1 input file generation using SQL scripts and the parameter will be "Feed name" & Number of Rows to populate Data
Req3	Write SQL script to identify the duplicate (rows) in each of the table Feed-1, 2, 3
Req4	Write the duplicate records in output file - "duplicates"
Req5	Create a script to replace all the duplicates with Unique rows and update back to respective Feed table
Req6	Execute the duplicate script and check the output is zero
Req7	Create SQL script to compare data from Feed-2,3 to Feed-1 and write in output file on the compared results
Req8	Create Test plan with all kinds of manual test cases in order to test this End to End functionality
Req9	Automate the test cases (if possible) using any method but should be automated...
Req 10	Document everything in word with all screen grabs as proper Project Document

1. Project Overview

Introduction: This project aims to develop a robust and automated framework using SQL scripts to manage data quality for various "Feed" tables. The primary focus is on handling duplicate records, ensuring data uniqueness, and comparing different datasets to identify discrepancies.

Project Goals:

- To create reusable SQL scripts for generating test data.
- To accurately detect and report duplicate rows within specified tables.
- To remove duplicate records, leaving only unique rows.
- To compare data between different feeds and highlight any mismatched records.
- To automate the entire process for consistent and repeatable execution.

Technology Stack: The solution is developed using SQL (specifically, the provided code appears to be compatible with PostgreSQL and MySQL-like syntax for different sections) and a relational database management system.

2. Detailed Requirement Breakdown

Requirement 1: Generate Random Data Input Files

Description: This task involves creating three distinct tables—Feed-1, Feed-2, and Feed-3—with varying numbers of columns and rows.

Implementation:

- Feed-1: A table with 10 columns and 10 rows.
- Feed-2: A table with 15 columns and 15 rows.
- Feed-3: A table with 20 columns and 20 rows.

Requirement 2: Automate Data Generation

Description: Instead of manually running scripts for each feed, this requirement automates the data generation process using a stored procedure.

Implementation: A function named generate_feed is created to handle the dynamic creation and population of tables. The function accepts feed_name and the number of rows and columns as parameters, allowing for flexible table creation based on the specified requirements.

Requirement 3: Write SQL Script to Identify Duplicates

Description: This involves creating a query to find any rows that are exact duplicates across all columns in Feed-1, Feed-2, and Feed-3.

Implementation: The script uses the GROUP BY clause on all columns and applies HAVING COUNT(*) > 1 to filter for groups of rows that appear more than once.

Requirement 4: Write Duplicate Records to an Output File

Description: The duplicate rows identified in the previous step need to be exported into a file named "duplicates".

Implementation: The COPY command is used to export the results of the duplicate-finding query for each table (feed1, feed2, feed3) into separate CSV files.

Requirement 5: Replace Duplicates with Unique Rows

Description: The core data quality step involves removing all duplicate rows from each of the three feed tables.

Implementation: The script uses a DELETE statement in conjunction with a ROW_NUMBER() window function partitioned by all columns. This assigns a unique number to each row within a duplicate set, and any row with a number greater than 1 is considered a duplicate and is deleted.

Requirement 6: Execute and Verify Zero Duplicates

Description: After the duplicates have been removed, the system must verify that no duplicates remain.

Implementation: The duplicate-finding query from Requirement 3 is run again on each table. The expected output is a count of zero, confirming that the de-duplication was successful.

Requirement 7: Compare Data and Write Results

Description: This task requires comparing the data from Feed-2 and Feed-3 against Feed-1 and writing the differences to a file.

Implementation: LEFT JOIN queries are used to identify rows that exist in one table but not the other. The results are then exported to a CSV file to capture the differences between the feeds.

3. Test Plan and Automation Framework

Requirement 8: Create a Manual Test Plan

Description: This plan provides a structured approach to manually test the end-to-end functionality of the SQL scripts.

Test Objectives:

- Verify duplicate detection.
- Validate that duplicate records are correctly exported to a file.
- Ensure duplicates are replaced with unique rows.
- Confirm accurate data comparison between feeds.
- Test Scenarios: The plan includes scenarios for duplicate identification (Req3), export (Req4), removal and replacement (Req5, Req6), and feed comparison (Req7). It covers cases with and without duplicates to ensure the scripts handle all situations correctly.

Requirement 9: Automate the Test Cases

Description: The manual test cases are automated to enable a repeatable, end-to-end testing process.

Implementation: A series of stored procedures and tables are created to build an automated testing framework.

* test_runs and test_results tables track each test run and its outcomes (PASS/FAIL).
* Procedures such as sp_generate_feed and sp_dedup_table are used to set up the test environment.
* Procedures such as sp_find_duplicates and sp_compare_tables execute the core logic.
* The sp_run_suite procedure orchestrates the entire test flow, from generating data with duplicates to verifying that they have been removed and logging the results.

4. SQL Code

* Requirement 1
  * Feed-1 (10 columns, 10 rows):

```
DROP TABLE IF EXISTS feed1;
CREATE TABLE feed1 (
    id SERIAL PRIMARY KEY,
    col1 INT,
    col2 TEXT,
    col3 NUMERIC(10,2),
    col4 DATE,
    col5 BOOLEAN,
    col6 TEXT,
    col7 INT,
```

```
    col8 NUMERIC(8,3),
    col9 TIMESTAMP,
    col10 TEXT
);
INSERT INTO feed1 (col1, col2, col3, col4, col5, col6, col7, col8, col9, col10)
SELECT
    (random()*100)::INT,
    md5(random()::text),
    round(random()*1000, 2),
    CURRENT_DATE - ((random()*365)::INT),
    (random() > 0.5),
    substr(md5(random()::text), 1, 5),
    (random()*50)::INT,
    round(random()*500, 3),
    NOW() - ((random()*100000)::INT || ' seconds')::interval,
    substr(md5(random()::text), 1, 8)
FROM generate_series(1, 10);

   * Feed-2 (15 columns, 15 rows):
     DROP TABLE IF EXISTS feed2;
CREATE TABLE feed2 (
    id SERIAL PRIMARY KEY,
    col1 INT,
    col2 TEXT,
    col3 NUMERIC(10,2),
    col4 DATE,
    col5 BOOLEAN,
    col6 TEXT,
    col7 INT,
    col8 NUMERIC(8,3),
    col9 TIMESTAMP,
    col10 TEXT,
    col11 INT,
    col12 NUMERIC(12,4),
    col13 BOOLEAN,
    col14 DATE,
    col15 TEXT
);
INSERT INTO feed2 (col1, col2, col3, col4, col5, col6, col7, col8, col9, col10,
                   col11, col12, col13, col14, col15)
SELECT
    (random()*100)::INT,
    md5(random()::text),
    round(random()*1000, 2),
```

```
    CURRENT_DATE - ((random()*365)::INT),
    (random() > 0.5),
    substr(md5(random()::text), 1, 5),
    (random()*50)::INT,
    round(random()*500, 3),
    NOW() - ((random()*100000)::INT || ' seconds')::interval,
    substr(md5(random()::text), 1, 8),
    (random()*200)::INT,
    round(random()*9999, 4),
    (random() > 0.5),
    CURRENT_DATE - ((random()*1000)::INT),
    substr(md5(random()::text), 1, 12)
FROM generate_series(1, 15);

   * Feed-3 (20 columns, 20 rows):
     DROP TABLE IF EXISTS feed3;
CREATE TABLE feed3 (
    id SERIAL PRIMARY KEY,
    col1 INT,
    col2 TEXT,
    col3 NUMERIC(10,2),
    col4 DATE,
    col5 BOOLEAN,
    col6 TEXT,
    col7 INT,
    col8 NUMERIC(8,3),
    col9 TIMESTAMP,
    col10 TEXT,
    col11 INT,
    col12 NUMERIC(12,4),
    col13 BOOLEAN,
    col14 DATE,
    col15 TEXT,
    col16 INT,
    col17 NUMERIC(10,3),
    col18 TEXT,
    col19 BOOLEAN,
    col20 DATE
);
INSERT INTO feed3 (col1, col2, col3, col4, col5, col6, col7, col8, col9, col10,
                   col11, col12, col13, col14, col15, col16, col17, col18, col19, col20)
SELECT
    (random()*100)::INT,
    md5(random()::text),
```

```
    round(random()*1000, 2),
    CURRENT_DATE - ((random()*365)::INT),
    (random() > 0.5),
    substr(md5(random()::text), 1, 5),
    (random()*50)::INT,
    round(random()*500, 3),
    NOW() - ((random()*100000)::INT || ' seconds')::interval,
    substr(md5(random()::text), 1, 8),
    (random()*200)::INT,
    round(random()*9999, 4),
    (random() > 0.5),
    CURRENT_DATE - ((random()*1000)::INT),
    substr(md5(random()::text), 1, 12),
    (random()*500)::INT,
    round(random()*12345, 3),
    substr(md5(random()::text), 1, 15),
    (random() > 0.5),
    CURRENT_DATE + ((random()*500)::INT)
FROM generate_series(1, 20);

 * Requirement 2
   DROP FUNCTION IF EXISTS generate_feed(feed_name text, num_rows int, num_cols int);
CREATE OR REPLACE FUNCTION generate_feed(feed_name text, num_rows int, num_cols
int)
RETURNS void AS
$$
DECLARE
    col_defs TEXT := '';
insert_cols TEXT := '';
    i INT;
    dyn_sql TEXT;
BEGIN
    -- Build dynamic CREATE TABLE statement
    col_defs := 'id SERIAL PRIMARY KEY';
FOR i IN 1..num_cols LOOP
        col_defs := col_defs || ', col' || i || ' TEXT';
        insert_cols := insert_cols || 'col' || i;
IF i < num_cols THEN
            insert_cols := insert_cols || ', ';
        END IF;
    END LOOP;
    dyn_sql := 'DROP TABLE IF EXISTS ' || feed_name || '; ' || 'CREATE TABLE ' || feed_name || '
(' || col_defs || ');';
    EXECUTE dyn_sql;
```

```
-- Populate with random data
    dyn_sql := 'INSERT INTO ' || feed_name || '(' || insert_cols || ') ' || 'SELECT ';
    FOR i IN 1..num_cols LOOP
        dyn_sql := dyn_sql || 'substr(md5(random()::text),1,8)';
        IF i < num_cols THEN
            dyn_sql := dyn_sql || ', ';
        END IF;
    END LOOP;
    dyn_sql := dyn_sql || ' FROM generate_series(1,' || num_rows || ');';
    EXECUTE dyn_sql;
END;
$$ LANGUAGE plpgsql;
-- Feed-1 with 10 cols × 10 rows
SELECT generate_feed('feed1', 10, 10);
-- Feed-2 with 15 cols × 15 rows
SELECT generate_feed('feed2', 15, 15);
-- Feed-3 with 20 cols × 20 rows
SELECT generate_feed('feed3', 20, 20);

 * Requirement 3
   -- Feed-1
SELECT col1, col2, col3, col4, col5, col6, col7, col8, col9, col10,
       COUNT(*) AS dup_count
FROM feed1
GROUP BY col1, col2, col3, col4, col5, col6, col7, col8, col9, col10
HAVING COUNT(*) > 1;
-- Feed-2
SELECT col1, col2, col3, col4, col5, col6, col7, col8, col9, col10,
       col11, col12, col13, col14, col15,
       COUNT(*) AS dup_count
FROM feed2
GROUP BY col1, col2, col3, col4, col5, col6, col7, col8, col9, col10,
         col11, col12, col13, col14, col15
HAVING COUNT(*) > 1;
-- Feed-3
SELECT col1, col2, col3, col4, col5, col6, col7, col8, col9, col10,
       col11, col12, col13, col14, col15, col16, col17, col18, col19, col20,
       COUNT(*) AS dup_count
FROM feed3
GROUP BY col1, col2, col3, col4, col5, col6, col7, col8, col9, col10,
         col11, col12, col13, col14, col15, col16, col17, col18, col19, col20
HAVING COUNT(*) > 1;

 * Requirement 4
```

```
  -- duplicates from feed1
COPY (
   SELECT col1, col2, col3, col4, col5, col6, col7, col8, col9, col10,
        COUNT(*) AS dup_count
   FROM feed1
   GROUP BY col1, col2, col3, col4, col5, col6, col7, col8, col9, col10
   HAVING COUNT(*) > 1
) TO '/tmp/duplicates_feed1.csv' WITH CSV HEADER;
-- duplicates from feed2
COPY (
   SELECT col1, col2, col3, col4, col5, col6, col7, col8, col9, col10,
        col11, col12, col13, col14, col15,
        COUNT(*) AS dup_count
   FROM feed2
   GROUP BY col1, col2, col3, col4, col5, col6, col7, col8, col9, col10,
         col11, col12, col13, col14, col15
   HAVING COUNT(*) > 1
) TO '/tmp/duplicates_feed2.csv' WITH CSV HEADER;
-- duplicates from feed3
COPY (
   SELECT col1, col2, col3, col4, col5, col6, col7, col8, col9, col10,
        col11, col12, col13, col14, col15, col16, col17, col18, col19, col20,
        COUNT(*) AS dup_count
   FROM feed3
   GROUP BY col1, col2, col3, col4, col5, col6, col7, col8, col9, col10,
         col11, col12, col13, col14, col15, col16, col17, col18, col19, col20
   HAVING COUNT(*) > 1
) TO '/tmp/duplicates_feed3.csv' WITH CSV HEADER;

 * Requirement 5
  -- Removing duplicates from Feed-1
DELETE FROM feed1 f
USING (
   SELECT id
   FROM (
      SELECT id,
           ROW_NUMBER() OVER (PARTITION BY col1, col2, col3, col4, col5, col6, col7, col8,
col9, col10
                        ORDER BY id) AS rn
      FROM feed1
   ) t
   WHERE t.rn > 1
) d
WHERE f.id = d.id;
```

```
-- Remove duplicates from Feed-2
DELETE FROM feed2 f
USING (
   SELECT id
   FROM (
      SELECT id,
           ROW_NUMBER() OVER (PARTITION BY col1, col2, col3, col4, col5, col6, col7, col8,
col9, col10,
                                col11, col12, col13, col14, col15
                       ORDER BY id) AS rn
      FROM feed2
   ) t
   WHERE t.rn > 1
) d
WHERE f.id = d.id;
-- Remove duplicates from Feed-3
DELETE FROM feed3 f
USING (
   SELECT id
   FROM (
      SELECT id,
           ROW_NUMBER() OVER (PARTITION BY col1, col2, col3, col4, col5, col6, col7, col8,
col9, col10,
                                col11, col12, col13, col14, col15, col16, col17, col18, col19, col20
                       ORDER BY id) AS rn
      FROM feed3
   ) t
   WHERE t.rn > 1
) d
WHERE f.id = d.id;

 * Requirement 6
  -- Checking duplicates in Feed-1
SELECT COUNT(*) AS dup_count
FROM (
   SELECT col1, col2, col3, col4, col5, col6, col7, col8, col9, col10
   FROM feed1
   GROUP BY col1, col2, col3, col4, col5, col6, col7, col8, col9, col10
   HAVING COUNT(*) > 1
) t;
-- Checking duplicates in Feed-2
SELECT COUNT(*) AS dup_count
FROM (
   SELECT col1, col2, col3, col4, col5, col6, col7, col8, col9, col10,
```

```
        col11, col12, col13, col14, col15
    FROM feed2
    GROUP BY col1, col2, col3, col4, col5, col6, col7, col8, col9, col10,
             col11, col12, col13, col14, col15
    HAVING COUNT(*) > 1
) t;
-- Checking duplicates in Feed-3
SELECT COUNT(*) AS dup_count
FROM (
    SELECT col1, col2, col3, col4, col5, col6, col7, col8, col9, col10,
        col11, col12, col13, col14, col15, col16, col17, col18, col19, col20
    FROM feed3
    GROUP BY col1, col2, col3, col4, col5, col6, col7, col8, col9, col10,
             col11, col12, col13, col14, col15, col16, col17, col18, col19, col20
    HAVING COUNT(*) > 1
) t;

 * Requirement 7
   -- Rows in Feed-2 but not in Feed-1
SELECT 'Feed2_ONLY' AS source, f2.*
FROM feed2 f2
LEFT JOIN feed1 f1
ON f2.col1 = f1.col1
AND f2.col2 = f1.col2
AND f2.col3 = f1.col3
WHERE f1.col1 IS NULL;
-- Rows in Feed-1 but not in Feed-2
SELECT 'Feed1_ONLY' AS source, f1.*
FROM feed1 f1
LEFT JOIN feed2 f2
ON f1.col1 = f2.col1
AND f1.col2 = f2.col2
AND f1.col3 = f2.col3
WHERE f2.col1 IS NULL;
-- Rows in Feed-3 but not in Feed-1
SELECT 'Feed3_ONLY' AS source, f3.*
FROM feed3 f3
LEFT JOIN feed1 f1
ON f3.col1 = f1.col1
AND f3.col2 = f1.col2
AND f3.col3 = f1.col3
WHERE f1.col1 IS NULL;
-- Rows in Feed-1 but not in Feed-3
SELECT 'Feed1_ONLY' AS source, f1.*
```

```
FROM feed1 f1
LEFT JOIN feed3 f3
ON f1.col1 = f3.col1
AND f1.col2 = f3.col2
AND f1.col3 = f3.col3
-- add as many as needed
WHERE f3.col1 IS NULL;
-- Feed-2 vs Feed-1 and exporting differences
COPY (
   SELECT 'Feed2_ONLY' AS source, f2.*
   FROM feed2 f2
   LEFT JOIN feed1 f1
   ON f2.col1 = f1.col1 AND f2.col2 = f1.col2
   WHERE f1.col1 IS NULL
   UNION ALL
   SELECT 'Feed1_ONLY' AS source, f1.*
   FROM feed1 f1
   LEFT JOIN feed2 f2
   ON f1.col1 = f2.col1 AND f1.col2 = f2.col2
   WHERE f2.col1 IS NULL
) TO '/tmp/feed2_vs_feed1_diff.csv' CSV HEADER;
-- Feed-3 vs Feed-1 and exporting differences
COPY (
   SELECT 'Feed3_ONLY' AS source, f3.*
   FROM feed3 f3
   LEFT JOIN feed1 f1
   ON f3.col1 = f1.col1 AND f3.col2 = f1.col2
   WHERE f1.col1 IS NULL
   UNION ALL
   SELECT 'Feed1_ONLY' AS source, f1.*
   FROM feed1 f1
   LEFT JOIN feed3 f3
   ON f1.col1 = f3.col1 AND f1.col2 = f3.col2
   WHERE f3.col1 IS NULL
) TO '/tmp/feed3_vs_feed1_diff.csv' CSV HEADER;

 * Requirement 9
  CREATE TABLE IF NOT EXISTS test_runs (
 run_id       BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
 started_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
 finished_at  TIMESTAMP NULL,
 overall_status VARCHAR(10) DEFAULT 'PENDING',
 notes        TEXT
);
```

```
CREATE TABLE IF NOT EXISTS test_results (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  run_id      BIGINT UNSIGNED NOT NULL,
  case_id     VARCHAR(20) NOT NULL,
  step_no     INT NOT NULL,
  test_name   VARCHAR(255) NOT NULL,
  expected    TEXT,
  actual      TEXT,
  status      VARCHAR(10) NOT NULL,
  details     TEXT,
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  KEY (run_id),
  CONSTRAINT fk_results_run FOREIGN KEY (run_id) REFERENCES test_runs(run_id) ON
DELETE CASCADE
);
CREATE TABLE IF NOT EXISTS duplicates_log (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  run_id      BIGINT UNSIGNED NOT NULL,
  table_name  VARCHAR(255) NOT NULL,
  group_hash  VARCHAR(64) NOT NULL,
  group_size  INT NOT NULL,
  sample      JSON,
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  KEY (run_id)
);
CREATE TABLE IF NOT EXISTS compare_results (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  run_id      BIGINT UNSIGNED NOT NULL,
  left_table  VARCHAR(255) NOT NULL,
  right_table VARCHAR(255) NOT NULL,
  side        VARCHAR(12) NOT NULL,
  row_hash    VARCHAR(64) NOT NULL,
  row_json    JSON,
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  KEY (run_id)
);
DELIMITER $$
CREATE OR REPLACE PROCEDURE sp_log_result(
  IN p_run_id BIGINT UNSIGNED,
  IN p_case   VARCHAR(20),
  IN p_step   INT,
  IN p_name   VARCHAR(255),
  IN p_exp    TEXT,
  IN p_act    TEXT,
```

```
  IN p_pass   BOOLEAN,
  IN p_details TEXT
)
BEGIN
  INSERT INTO test_results(run_id, case_id, step_no, test_name, expected, actual, status,
details)
  VALUES(p_run_id, p_case, p_step, p_name, p_exp, p_act, IF(p_pass,'PASS','FAIL'),
p_details);
END$$
CREATE OR REPLACE PROCEDURE sp_generate_feed(
  IN p_table    VARCHAR(255),
  IN p_num_cols INT,
  IN p_num_rows INT,
  IN p_dup_rows INT
)
BEGIN
  DECLARE i INT DEFAULT 1;
DECLARE col_defs TEXT DEFAULT 'row_id BIGINT UNSIGNED NOT NULL
AUTO_INCREMENT PRIMARY KEY';
  DECLARE col_list TEXT DEFAULT '';
DECLARE expr_list TEXT DEFAULT '';
  DECLARE col_name VARCHAR(255);
  DECLARE typ INT;
SET @sql := CONCAT('DROP TABLE IF EXISTS `', p_table, '`');
  PREPARE s FROM @sql; EXECUTE s; DEALLOCATE PREPARE s;
WHILE i <= p_num_cols DO
    SET typ = ((i - 1) % 5) + 1;
CASE typ
      WHEN 1 THEN SET col_name = CONCAT('c', i, '_text');
SET col_defs = CONCAT(col_defs, ', `', col_name, '` VARCHAR(50)');      SET expr_list =
CONCAT(expr_list, "CONCAT('T', LPAD(FLOOR(RAND()*999999),6,'0'))");
WHEN 2 THEN SET col_name = CONCAT('c', i, '_int');    SET col_defs = CONCAT(col_defs, ',
`', col_name, '` INT');
SET expr_list = CONCAT(expr_list, "FLOOR(RAND()*1000)");
      WHEN 3 THEN SET col_name = CONCAT('c', i, '_date');
SET col_defs = CONCAT(col_defs, ', `', col_name, '` DATE');            SET expr_list =
CONCAT(expr_list, "DATE_ADD('2020-01-01', INTERVAL FLOOR(RAND()*1825) DAY)");
WHEN 4 THEN SET col_name = CONCAT('c', i, '_num');    SET col_defs = CONCAT(col_defs, ',
`', col_name, '` DECIMAL(10,2)');
SET expr_list = CONCAT(expr_list, "ROUND(RAND()*10000,2)");
      WHEN 5 THEN SET col_name = CONCAT('c', i, '_bool');
SET col_defs = CONCAT(col_defs, ', `', col_name, '` TINYINT(1)');       SET expr_list =
CONCAT(expr_list, "FLOOR(RAND()*2)");
    END CASE;
```

```
SET col_list = CONCAT(col_list, '`', col_name, '`');
    IF i < p_num_cols THEN
      SET col_list =
```